

AI assistance log

What the AI got wrong while building this, and how each problem was fixed

Valsoft AI Engineer assessment, supporting evidence ArcVault Intake & Triage

I built this with Claude Code (Claude Opus) as a pair programmer. This log records, as they happened, the wrong assumptions, bugs and bad outputs, and how each was fixed. It records what went wrong, not what went well.

#	Stage	What went wrong / wrong assumption	How it was caught	Fix
1	Setup	The original plan assumed Docker, Python and Git were installed. None were, and Docker Desktop needs WSL2 plus a reboot.	Checked the tooling before writing any code.	I switched to n8n Cloud and installed Python, Git and Node with winget. On Cloud, <code>\$env</code> , the local filesystem and the n8n CLI aren't available, so secrets moved to an n8n credential, persistence moved to Webhook.site plus the webhook response, and import moved to the UI.
2	Setup	Reading the brief .docx failed ("file is being used by another process") because it was open in Word.	Error on read.	Opened it with shared-read access instead.
3	Setup	After the winget install, <code>node / git</code> weren't found in the same shell.	Command not found.	PATH is only picked up by new shells, so I refreshed it from the registry.
4	Rules design	The first keyword list for <code>security_signal</code> included <code>login</code> , <code>logging in</code> and <code>sso</code> . Combined with "raise priority to High", that would have made message #4 (a how-to question about SSO) High priority.	Tracing message #4 through the rules by hand before writing tests.	Narrowed the list to access <i>failures</i> only (401/403, "can't log in", "locked out"...). Unit test #4 SSO question pins this.
5	Rules design	The first <code>outage_language</code> list included bare <code>everyone</code> and <code>all users</code> . Feature requests use those words too ("everyone on my team wants..."), so they would have been escalated as outages.	Review of false-positive risk.	Removed them. Kept multi-word phrases only, matched on word boundaries.
6	Rules design	The spec I started from said to escalate billing when "either the invoiced amount or the discrepancy exceeds \$500". That collapses to "invoiced amount > \$500", because the discrepancy can never be larger than the largest amount. The "either" option added nothing.	Writing the threshold code.	Renamed the option to <code>any_amount</code> (chosen) vs <code>discrepancy</code> . Both are unit-tested. Amounts are parsed from the raw text by regex, not taken from the LLM, so a hallucinated number can't trigger escalation.

#	Stage	What went wrong / wrong assumption	How it was caught	Fix
7	n8n glue	I nearly used <code>\$json</code> inside Code nodes. In n8n's default "Run Once for All Items" mode <code>\$json</code> isn't defined; <code>\$input.first().json</code> is.	Design review. <code>tests/workflow_smoke.test.js</code> now runs the generated Code nodes with only <code>\$input/\$()</code> available, so this class of bug fails the test.	Used <code>\$input.first().json</code> and <code>\$('Node').first().json</code> throughout.
8	Git	Git had no user identity configured on this machine.	<code>git config user.email</code> returned nothing.	Set a repo-local identity only. Didn't touch global config.
9	LLM	I had checked Groq's model docs and they still listed <code>llama-3.3-70b-versatile</code> , so I set it as the default. On a free-tier key the API returned <code>404 model_not_found</code> .	First live call.	Listed the models via <code>/openai/v1/models</code> and switched to <code>openai/gpt-oss-120b</code> (reasoning model, <code>reasoning_effort: low</code>). The model name lives only in <code>config/rules.json</code> . Lesson: check model availability with the actual key, not the docs page.
10	LLM	I assumed the free tier had enough throughput. <code>gpt-oss-120b</code> is capped at 8,000 tokens/minute , and one message (3 calls, ~2k tokens each) nearly uses all of it, so back-to-back test calls hit 429.	429 responses during prompt testing.	<code>check_groq.py</code> now waits for the "try again in Xs" time. <code>send_samples.py / eval.py</code> pace messages. This is also a production-scale point in the write-up: shorter prompts, a smaller model for classification, paid tier.
11	Prompts	v1 classification reported confidence 0.92 on a three-intent message, despite the calibration guide.	Edge case E1.	Added <code>secondary_categories</code> plus a deterministic <code>multi_intent</code> escalation rule. See <code>CHANGELOG_PROMPTS.md</code> .
12	Prompts	Angry \$45 billing complaint gets High priority, even after v2 said "never by tone".	Edge case E5, before and after v2.	Not fixed; documented as a known disagreement. It doesn't change routing.
13	n8n setup	The first <code>N8N_WEBHOOK_URL</code> was the Webhook node's test URL (<code>/webhook-test/...</code>), which returns 404 unless "Execute workflow" was just clicked.	First live POST returned 404 with n8n's hint.	Switched to the production URL (<code>/webhook/...</code>); the workflow must be Activated. The README calls this out.
14	Tooling	I used a PowerShell <code>Get-Content / Set-Content</code> one-liner to change a default value. Windows PowerShell 5.1 read the UTF-8 file as ANSI, so an en dash in <code>eval.py</code> turned into <code>â€‘</code> , and a BOM was added.	The editor's diff of the change showed the mojibake.	Replaced the character, rewrote both files as UTF-8 without BOM, and checked them for non-ASCII bytes. After that I only edited files with the editor tool, not shell text replacement.

#	Stage	What went wrong / wrong assumption	How it was caught	Fix
15	Git	My pre-commit "no Groq key" check used invalid <code>git grep</code> argument order. It errored, and the PowerShell <code>if</code> read the error as "no match", so the commit went ahead without a real check.	Read the command output after the commit.	Re-ran it correctly against <code>HEAD</code> (0 hits, <code>.env</code> not tracked). The same key check is also built into <code>build_workflow.py</code> , which refuses to write the workflow if it finds a key. I made the same mistake again when packaging the submission. I caught it the same way and re-checked the pushed commit and the full history (0 hits). The real fix would be a pre-commit hook instead of a hand-typed command.
16	n8n setup	My import instructions said "create a Header Auth credential". In the n8n Cloud UI the imported nodes didn't show a credential dropdown where the instructions said it would, and the Enrich node was left without a credential. Real messages then returned <code>200</code> with an empty body, and later <code>Credentials not found</code> on enrichment.	Live test: the empty-message path (1 LLM call) worked, the full path didn't. I split the test by path to narrow it down, then read the error the workflow had recorded in <code>llm_errors</code> .	Switched the generator and README to the built-in Bearer Auth credential type, which is what the user found and set up in the UI. The <code>Credentials not found</code> run is a good demonstration of the failure path: the record was escalated as <code>llm_failure</code> with the reason, not dropped.
17	Testing	My ad-hoc PowerShell test command put <code>\$1,240</code> inside a double-quoted string, so PowerShell interpolated <code>\$1 / \$980</code> and the workflow received <code>\,240</code> .	The returned <code>raw_message</code> looked wrong.	Not a pipeline bug. The real scripts send text from <code>tests/golden.json</code> , so they aren't affected. I only used the scripts after that.
18	Instructions	I gave the user a <code>curl.exe -d "{\"source\":...}"</code> command to run by hand. That escaping works in cmd and bash, but Windows PowerShell 5.1 strips the <code>\</code> when calling native executables, so curl got broken JSON (n8n returned 422) and treated the leftover words as hostnames.	The user pasted the error output.	Gave an <code>Invoke-RestMethod</code> version instead, and added <code>send_samples.py --message "..."</code> <code>--source ...</code> so there's nothing to quote by hand. The README notes the pitfall.
19	Prompts	The v2 prompt changes (aimed at E1, E5 and S1) raised confidence on the vague message E2 "it's broken again" from 0.62 (v1) to 0.78, so it no longer escalates and goes straight to engineering. I didn't re-test E2 after v2, only the cases I had changed.	Full eval run (<code>output/eval_report.md</code>).	Recorded as a failing case. Lesson: after any prompt change, re-run the whole eval set, not just the cases the change targeted.

#	Stage	What went wrong / wrong assumption	How it was caught	Fix
20	Rules	The <code>security_signal</code> detail string always said "priority raised to High", even when the LLM had already returned High and nothing changed (sample #1). The structured field <code>priority_adjusted_by</code> was correct (<code>null</code>); only the text was wrong. The unit test only covered the case where the priority does get raised.	Reviewing the rendered screenshot of sample #1.	The detail now says "already High, no change" or "raised from X to High". Added a unit test for the already-High case. workflow_version 1.0.1; redeployed and re-ran the samples.
21	Git	Making the repository public: I rewrote history to remove a personal Webhook.site capture URL, then ran the force-push and the visibility change in one command. The push was rejected (<code>stale info</code> , because the rewrite had moved the local <code>origin/main</code> ref) but the visibility change still ran, so the repository was briefly public with the old history.	Read the command output: the push said ! [rejected] while the repo already read <code>PUBLIC</code> .	Force-pushed immediately and confirmed the remote no longer contains the URL. The sequencing was the real mistake: the irreversible, outward-facing step should have been a separate command after the push was verified. GitHub can keep unreachable objects addressable by SHA, so the capture URL is also being rotated.